More practice - Recursion and backtracking

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Recursion and backtracking | 1 questions | tap a title to jump to it

H 1. Matrix Flower Placement with House Adjacency **1Point3Acres 2026-05-31**

STUDY PATH: question 147 of 167 (more practice)

Priority score 2.5 | Recursion and backtracking | asked 142 days ago | frequency Low

HARD | 1Point3Acres | onsite-coding | 1 of 1

Matrix Flower Placement with House Adjacency

ASKED 2026-05-31 | 142 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 190/228**

ABOUT THIS QUESTION

Topic: More practice - Recursion and backtracking

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-05-31 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/matrix-flower-placement-with-house-adjacency

QUESTION

EXAMPLES

```
H 0 0 0
H 0 0 0
0 0 0 H
0 0 0 0
```

The task is to fill some 0 cells with F while satisfying both the row/column constraint and each house's adjacency requirement.

APPROACH

Practice N-Queens with row/column sets and a recursive placement order.
Drill grid-neighbor helpers and constraint pruning before writing the full DFS.
Prepare the boolean version first, then generalize to collect all solutions.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether each row and column holds at most one or exactly one flower, whether a house needs at least one or exactly one adjacent flower, whether adjacency is four-way, and whether to return one solution, a boolean, or all.

BRUTE FORCE TO SAY OUT LOUD

Try every subset of empty cells. Exponential.

OPTIMAL IDEA

Row-by-row backtracking like N-Queens: place at most one flower per row, track used columns, and check each house as soon as the row below it is decided (its neighbors are in rows h - 1 to h + 1). Prune at the first failed house. The boolean version stops at the first solution; the all-solutions version continues.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

No houses, a house with no empty neighbor (impossible), two houses sharing one neighbor, a 1 x 1 grid.

LIKELY FOLLOW-UPS

Exactly-one rule, all solutions, or a faster exact-cover search.

PITFALLS

Checking a house before the next row is decided. Not restoring the cell on backtrack.

HOW TO THINK ABOUT IT

At most one flower per row and column is the N-Queens pattern: decide row by row which column (if any) gets the flower and remember the used columns. The extra rule about houses is local, so check a house as soon as the last row that can affect it is decided, and prune the branch if it fails.

Basis: N-Queens style backtracking (LeetCode 51/52) with extra local constraints. Assumptions stated explicitly: every house must have at least one flower in a 4-neighbour cell; rows and columns hold at most one flower (or exactly one if requested).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    empty = cells_with(grid, 0)
    count = 0
    for k in range(len(empty) + 1):
        for flowers in combinations(empty, k):
            if not one_per_row_and_col(flowers):
                continue
            if houses_covered(grid, flowers):
                count += 1
    return count
```

TIME	SPACE
O(2^cells)	O(cells)

WHY IT IS SLOW

2^cells subsets: exponential. Backtracking row by row with a used-columns set and early house checks prunes almost all of them (still exponential in the worst case).

OPTIMAL SOLUTION · Row-by-row backtracking with used-column set and delayed house checks

TIME	SPACE
exponential worst case, h... exponential worst case, heavily pruned	O(rows + cols)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What changes between 'at most one' and 'exactly one' flower per row and column?

With 'exactly one' the number of flowers is forced (rows must equal columns) and a row cannot be skipped, which prunes much harder, like N-Queens without the diagonal rule. With 'at most one' you also branch on skipping a row. State which reading you are using before writing code.

FOLLOW-UP 2 How would you speed up the search?

Prune earlier by checking each house as soon as its last neighbour row is decided (done above), order houses by how few candidate cells they have (most-constrained first), and keep bitmasks for used columns. For large grids this becomes a constraint satisfaction or exact-cover problem, so a SAT solver or DLX is the next step. **Cost:** the worst case stays exponential; most-constrained-first ordering and early house checks cut the explored states by orders of magnitude on typical grids.

FOLLOW-UP 3 Exactly one flower per row and column.

Add a flag that forbids skipping a row and requires as many columns as rows (otherwise no solution). The pruning is stronger, since every row must find a column.

FOLLOW-UP 4 Count the solutions instead of listing them.

Keep the same recursion but add one to a counter at the leaf instead of copying the grid; with a bitmask of used columns and memoisation on (row, mask, pending-house state) it can become a DP.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Matrix Flower Placement with House
# Adjacency
#
# Examples: H 0 0 0 H 0 0 0 0 0 0 H 0 0 0 0 The task is to
# fill some 0 cells with F while satisfying both the
# row/column constraint and each house's adjacency
# requirement.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What changes between 'at most one' and 'exactly one'
# flower per row and column?
# 2. How would you speed up the search?
# 3. Exactly one flower per row and column.
# 4. Count the solutions instead of listing them.
#
# INTERVIEW NOTES
# At most one flower per row and column is the N-Queens
# pattern: decide row by row which column (if any) gets
# the flower and remember the used columns. The extra rule
# about houses is local, so check a house as soon as the
# last row that can affect it is decided, and prune the
# branch if it fails.
#
# BRUTE FORCE (say it first): Choose any subset of empty
# cells and check all the rules
# Enumerate every subset of empty cells, keep those with
# at most one flower per row and column whose every house
# has a flower neighbour, and count them. Exact, the
# oracle, and exponential in the number of cells. Time
# O(2^cells). Space O(cells).


import itertools


# BRUTE FORCE (say it first): Choose any subset of empty
# cells and check all the rules | time O(2^cells)
def count_placements_brute(grid, exactly_one=False):
    R, C = len(grid), len(grid[0])
    empties = [
        (r, c)
        for r in range(R)
        for c in range(C)
        if grid[r][c] == "0"
    ]
    total = 0
    for k in range(len(empties) + 1):
        for chosen in itertools.combinations(
            empties, k
        ):
            rows, cols = [r for r, _ in chosen], [
                c for _, c in chosen
            ]
            if len(set(rows)) != len(rows) or len(
                set(cols)
            ) != len(cols):
                continue
            if exactly_one and not (
                len(rows) == R == C
            ):
                continue
            fl = set(chosen)
            if all(
                any(
                    (r + dr, c + dc) in fl
                    for dr, dc in (
                        (1, 0),
                        (-1, 0),
                        (0, 1),
                        (0, -1),
                    )
                )
                for r in range(R)
                for c in range(C)
                if grid[r][c] == "H"
            ):
                total += 1
    return total


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# BASE QUESTION (optimal): Row-by-row backtracking with
# used-column set and delayed house checks
# Idea: Place at most one flower per row, tracking used
# columns, and only on empty cells ('0').
def place_flowers(grid, exactly_one=False,
                  find_all=False):
    # 'H' house, '0' empty. At most one F per row and
    # column; every house needs an F on a side.
    R, C = len(grid), len(grid[0])
```

```
    cells = [list(row) for row in grid]
    houses = [(r, c) for r in range(R) for c in range(C)
              if cells[r][c] == "H"]
    found = []

    def house_ok(r, c):
        # is some neighbour of this house a flower?
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            if 0 <= a < R and 0 <= b < C \
                    and cells[a][b] == "F":
                return True
        return False

    def row_houses_ok(r):
        return all(house_ok(hr, hc)
                   for hr, hc in houses if hr == r)

    def solve(r, used):
        # FOLLOW-UP 4: find_all=True lists every solution
        if found and not find_all:
            return
        # rows up to r-1 are decided, so row r-2 is final
        if r >= 2 and not row_houses_ok(r - 2):
            return
        if r == R:
            # FOLLOW-UP 3: exactly_one=True: a flower in
            # every column
            full = len(used) == C
            if row_houses_ok(R - 1) and (full or
                                         not exactly_one):
                found.append(["".join(x) for x in cells])
            return
        # try a flower in every free column of this row
        for c in range(C):
            if c not in used and cells[r][c] == "0":
                cells[r][c] = "F"
                solve(r + 1, used | {c})
                cells[r][c] = "0"
        # or leave the row empty (not allowed for exactly
        # one)
        if not exactly_one:
```

```
            solve(r + 1, used)

    solve(0, frozenset())
    if find_all:
        return found
    return found[0] if found else None


# FOLLOW-UP 4: Count the solutions instead of listing
# them.
# Answer: Keep the same recursion but add one to a counter
# at the leaf instead of copying the grid; with a bitmask
# of used columns and memoisation on (row, mask, pending-
# house state) it can become a DP.
def count_solutions(grid, exactly_one=False):
    return len(place_flowers(grid, exactly_one, True))


if __name__ == "__main__":
    grid = ["H00", "0H0", "00H"]
    sol = place_flowers(grid)
    assert sol is not None
    # every house has a flower beside it
    for r in range(3):
        for c in range(3):
            if grid[r][c] == "H":
                near = []
                for a, b in STEPS:
                    x, y = r + a, c + b
                    if 0 <= x < 3 and 0 <= y < 3:
```

```
                        near.append(sol[x][y])
                assert "F" in near
    # one flower per row and column
    assert all(row.count("F") <= 1 for row in sol)
    for c in range(3):
        assert [row[c] for row in sol].count("F") <= 1
    # a lone house in a 1 x 1 grid cannot be served
    assert place_flowers(["H"]) is None
    assert count_solutions(["00", "00"], True) == 2
    assert place_flowers(["000", "000"], True) is None
```

NOTES FROM CANDIDATES

This is an N-Queens-style DFS / backtracking problem with extra local constraints around houses.
Track occupied rows and columns, then validate the four-neighbor requirement either incrementally or at the leaf.
The two variants differ on at most one vs exactly one flower per row/column; state the assumption before coding.

PREPARATION

Practice N-Queens with row/column sets and a recursive placement order.
Drill grid-neighbor helpers and constraint pruning before writing the full DFS.
Prepare the boolean version first, then generalize to collect all solutions.